

Autómatas, Teoría de Lenguajes y Compiladores

(72.39)

Proyecto Especial

Diseño e Implementación de un Lenguaje

Instituto Tecnológico de Buenos Aires



Grupo G123

Integrantes:

- Castelnuovo, Carolina (65539)
- Cecotto, Florencia (65068)
- Krens, Facundo (65049)
- Orbaj, Dalila (65471)

Docentes:

- Arias Roig, Ana María
- Golmar, Mario Agustín

Fecha de Entrega: 14/06/2026

2026 - 1Q

Tabla de Contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1 Dominio.....	4
2.2 Lenguaje.....	4
3. Implementación.....	5
3.1 Frontend.....	5
3.2 Backend.....	6
3.3 Dificultades Encontradas.....	7
4. Futuras Extensiones.....	8
5. Conclusiones.....	9
6. Referencias.....	9
7. Bibliografía.....	9

1. Introducción

El presente informe describe el diseño y la implementación de un lenguaje específico de dominio orientado a la administración de finanzas personales. La propuesta busca ofrecer una sintaxis simple, cercana al lenguaje natural, que permita registrar ingresos, gastos en un único pago y en cuotas, suscripciones, consultas, modificaciones y reportes sin que el usuario deba interactuar directamente con SQL ni con el esquema relacional subyacente.

El proyecto fue desarrollado sobre el compilador base provisto por la cátedra, utilizando Flex para el análisis léxico y Bison para el análisis sintáctico. A partir de dicho esqueleto se definió un conjunto propio de tokens, una gramática específica del dominio, acciones semánticas para construir el árbol de sintaxis abstracta, una etapa de análisis semántico y un generador de código que emite un script SQL para PostgreSQL.

La motivación principal del lenguaje es simplificar tareas habituales de seguimiento financiero que suelen realizarse de forma dispersa en planillas, notas o aplicaciones bancarias. En particular, el dominio elegido permite expresar operaciones con fechas absolutas o relativas, categorías, descripciones textuales, cuotas, recurrencias y formatos de reporte. De esta manera, el lenguaje actúa como una capa de abstracción entre el usuario y la persistencia de la información financiera.

2. Modelo Computacional

El modelo computacional propuesto se basa en la compilación de un programa compuesto por una o más sentencias financieras. Cada sentencia representa una acción concreta dentro del dominio: registrar una operación, cambiar el contexto de divisa, consultar información, editar un elemento ya existente, eliminarlo, finalizar una obligación recurrente o solicitar un reporte. El compilador recibe dichas sentencias como entrada, las *tokeniza*, valida su estructura gramatical, construye un AST, realiza validaciones semánticas y finalmente genera código SQL.

Desde el punto de vista conceptual, el lenguaje fue pensado para que el usuario escriba instrucciones imperativas de alto nivel. Por ejemplo, una sentencia como `gasto 15000 categoria supermercado descripción compras semanales` no describe cómo insertar una fila ni cómo relacionarla con otras tablas, sino qué operación quiere registrar el usuario. La traducción a SQL queda encapsulada dentro del compilador, que transforma esa intención en instrucciones ejecutables sobre PostgreSQL.

La salida del compilador se corresponde con un artefacto textual: un script SQL emitido por salida estándar. Dicho script incluye el esquema relacional necesario de forma idempotente y luego genera las sentencias específicas asociadas a cada instrucción del programa fuente. En consecuencia, el lenguaje no ejecuta directamente la base de datos, sino que produce una salida que puede ser ejecutada posteriormente en un entorno PostgreSQL.

2.1 Dominio

El dominio elegido corresponde a la gestión de finanzas personales de un único usuario. El lenguaje permite representar ingresos, gastos, compras en cuotas, suscripciones recurrentes y reportes periódicos. La elección de este dominio se justifica porque combina operaciones simples con elementos temporales más complejos, como fechas relativas, rangos de consulta, cuotas futuras y recurrencias mensuales, semanales o anuales.

Para mantener un alcance razonable, el proyecto no contempla integración con entidades bancarias externas, sincronización automática con cuentas reales ni conversión de divisas en tiempo real. La divisa se modela como contexto de las operaciones posteriores, mientras que las categorías se ingresan directamente dentro de cada operación y son normalizadas a minúsculas y sin tildes para evitar duplicaciones lógicas, como “alimentacion” y “alimentación”.

El lenguaje también contempla operaciones de mantenimiento sobre datos ya persistidos, como editar, eliminar o finalizar una operación. Estas construcciones fueron diseñadas alrededor de un identificador numérico, que es generado por PostgreSQL y luego utilizado por el usuario para referenciar operaciones previas. Dado que dichos identificadores pertenecen al estado de la base de datos, las validaciones de existencia y compatibilidad se delegan al runtime de SQL.

2.2 Lenguaje

El lenguaje de entrada se organiza como una secuencia de sentencias, donde cada sentencia comienza con una palabra reservada que indica su propósito. Las palabras reservadas no son sensibles a mayúsculas y minúsculas, y se contemplan variantes con y sin tildes, y con o sin “ñ” en términos relevantes como “categoría”, “descripción”, “suscripción” y “mañana” (también se contempla “maniana”). Además, se incorporó soporte para comentarios de línea y comentarios multilínea, lo cual facilita escribir programas de prueba más legibles.

Las principales construcciones implementadas a nivel sintáctico son las siguientes:

- **divisa** <ID>, para fijar la divisa activa de las operaciones siguientes.
- **gasto** <NUMERO>, con campos opcionales de cuotas, categoría, fecha y descripción.
- **ingreso** <NUMERO>, con campos opcionales de categoría, fecha y descripción.
- **suscripcion** <NUMERO> <frecuencia>, con categoría, fecha de inicio, fecha de fin y descripción opcionales.
- consultar período, donde el período puede ser mensual, semanal, anual o un rango desde / hasta.
- **editar** <NUMERO> <campo>, para modificar monto, categoría, fecha o descripción de una operación.
- **eliminar** <NUMERO> y **finalizar** <NUMERO>, para operar sobre identificadores existentes.
- **reporte** <formato> <período>, con formatos html, texto plano y pdf.

Además, el lenguaje permite expresar montos enteros o decimales. Para mantener una representación consistente con el dominio financiero, el compilador conserva hasta dos cifras luego de la coma o el punto decimal; en caso de ingresar más precisión, el valor se trunca. También se incorporó soporte para multiplicadores de magnitud, como K, M y B, que permiten

escribir montos grandes de forma más compacta. Por ejemplo, 15K representa 15000, 2.5M representa 2500000 y 1B representa 1000000000. Se admiten espacios y tabulaciones (por ejemplo, “1.5 M”) y concatenaciones (“1.5 K M”), siempre y cuando el valor numérico no supere al máximo permitido.

En el Cód. 2.2.1 se muestra un programa válido que combina varias construcciones del lenguaje. El ejemplo ilustra el estilo buscado: instrucciones breves, legibles y centradas en el dominio, sin exponer detalles de SQL ni de la estructura interna de datos.

```
// Fijar una divisa por defecto
divisa ARS
// Registrar operaciones
ingreso 200K categoria salario fecha hoy descripcion sueldo de abril
gasto 15000 categoria supermercado descripcion compras semanales
gasto 180000 cuotas 12 categoria electrodomesticos fecha 15-03-2026
descripcion lavarropas

// Consultar y reportar
consultar mensual
reporte html mensual
```

Código 2.2.1: Programa de ejemplo del lenguaje de finanzas personales.

3. Implementación

La implementación se realizó en lenguaje C, tomando como base el proyecto Flex-Bison-Compiler. La estructura del repositorio mantiene una separación clara entre frontend, backend, utilidades de soporte y casos de prueba. El flujo principal se encuentra en EntryPoint.c, donde se inicializan los módulos, se ejecuta el análisis sintáctico, luego el análisis semántico y, únicamente si ambas etapas resultan exitosas, se invoca la generación de código SQL.

El proyecto puede compilarse y ejecutarse mediante los scripts provistos en la carpeta src/main/bash. Además, se incluye una suite de pruebas bajo src/test/c, separada en casos de aceptación y rechazo. En total, el repositorio contiene 35 casos de aceptación y 18 casos de rechazo, cubriendo tanto entradas correctas como errores léxicos, sintácticos y semánticos esperados.

3.1 Frontend

En Flex se definieron los patrones léxicos para reconocer montos, identificadores, fechas, palabras reservadas, comentarios y descripciones. En el caso de los montos, se incorporó soporte para valores enteros, valores decimales y multiplicadores de magnitud como K, M y B. El compilador conserva hasta dos cifras decimales, de acuerdo con la representación utilizada para montos financieros; si el usuario escribe más de dos decimales, el valor se trunca antes de continuar con las siguientes etapas. En los montos con multiplicadores, este truncamiento se realiza antes de aplicar K, M o B.

Las fechas absolutas siguen el formato *dd-mm-yyyy*, mientras que las fechas relativas se reconocen mediante los literales “ayer”, “hoy” y “maniana”/“mañana”. También se definieron reglas específicas para aceptar palabras reservadas con tildes o sin tildes, manteniendo la misma representación interna.

Una decisión relevante fue el manejo de la descripción. A diferencia de otros campos, la descripción no requiere comillas: luego de reconocer la palabra reservada “description”, el scanner entra en un contexto especial que consume el resto de la línea como un único *String*. Esto mejora la usabilidad del lenguaje, aunque impone que la descripción sea el último campo de la sentencia en la línea correspondiente.

En Bison se definieron los terminales “NUMERO”, “STRING”, “DATE”, “ID” y las palabras reservadas del dominio. Como no terminales principales se definieron “program”, “sentences”, “sentence” y las sentencias específicas de cada construcción. La gramática también incluye no terminales auxiliares para campos opcionales, períodos, frecuencias, formatos de reporte, fechas y listas de campos editables.

Las acciones semánticas de Bison construyen nodos del AST (*Abstract Syntax Tree*) para cada sentencia reconocida. Por ejemplo, una sentencia de gasto crea un *ExpenseSentence* con el monto y los campos opcionales; una sentencia de reporte crea un *ReportSentence* con el formato y el período; y una sentencia de edición construye una lista de campos editables. Esta representación permite que las siguientes etapas del compilador no dependan del texto original, sino de una estructura tipada y específica del dominio.

Los casos de prueba de aceptación cubren registros simples, cambio de divisa, consultas por período, reportes en texto plano, HTML y PDF, gastos en cuotas, suscripciones, palabras con tildes, fechas bisiestas, múltiples sentencias, montos decimales, multiplicadores de magnitud y descripciones largas. Los casos de rechazo cubren programas malformados, montos inválidos, fechas inexistentes, rangos invertidos, cuotas no positivas, formatos no soportados y sentencias incompletas.

3.2 Backend

El backend implementado se divide en dos responsabilidades principales: análisis semántico y generación de código. La primera etapa recorre el AST y valida reglas propias del dominio que no conviene resolver únicamente desde la gramática. La segunda etapa toma el programa ya validado y emite un script SQL para PostgreSQL por salida estándar.

Durante el análisis semántico se construye una tabla global de símbolos. Dado que el DSL no posee scopes anidados, dicha tabla mantiene principalmente la divisa activa y el conjunto de categorías utilizadas, normalizadas en minúsculas y sin tildes. La divisa activa comienza con un valor por defecto ARS y se actualiza cada vez que aparece una sentencia divisa.

Las validaciones semánticas implementadas incluyen montos positivos para ingresos, gastos y suscripciones; cantidad de cuotas mayor que cero; fechas calendario válidas, incluyendo años bisiestos; rangos temporales consistentes para consultas y reportes; y suscripciones cuya fecha de finalización no sea anterior a la fecha de inicio. En cambio, la existencia de identificadores y la compatibilidad de una operación, al finalizarla, se dejan como validaciones de *runtime*, porque dependen de datos persistidos en PostgreSQL.

La generación de código produce primero un prólogo con DDL idempotente para crear, si no existen, las tablas operaciones, cuotas y suscripciones. Luego, por cada sentencia del programa fuente, se emite el SQL correspondiente. Las operaciones simples generan INSERT sobre operaciones; los gastos en cuotas generan una operación padre y cuotas futuras mediante *generate_series*; las suscripciones generan una operación padre y una fila asociada en suscripciones; las consultas y reportes generan SELECT filtrados por período; las ediciones, eliminaciones y finalizaciones generan UPDATE o DELETE según corresponda.

Para los reportes se contemplan tres formatos de salida: texto plano, HTML y PDF. El formato de texto plano genera una salida simple y legible con el detalle de las operaciones del período. El formato HTML construye una representación estructurada en forma de tabla, adecuada para ser abierta desde un navegador o reutilizada en otros contextos. Finalmente, el formato PDF se construye como un documento PDF 1.4 generado desde SQL puro. En todos los casos, el compilador emite un script SQL por salida estándar; al ejecutarse dicho script, el reporte se obtiene como resultado de un SELECT y, en el caso de usar psql, también puede persistirse automáticamente en un archivo con extensión .txt, .html o .pdf mediante meta-comandos del cliente.

Otra mejora incorporada en la generación fue el tratamiento de montos. El backend normaliza los valores numéricos recibidos desde el frontend, contemplando decimales de hasta dos cifras y multiplicadores como K, M y B. Esto permite que distintas formas de escritura del usuario se traduzcan a valores numéricos consistentes dentro del SQL generado y dentro de los reportes.

En el Cód. 3.2.1 se muestra, de manera simplificada, el tipo de salida que puede producir el generador para un gasto simple.

```
CREATE TABLE IF NOT EXISTS operaciones (...);

INSERT INTO operaciones (tipo, monto, divisa, categoria, fecha,
descripcion)
VALUES ('gasto', 15000, 'ARS', 'supermercado', DATE '2026-06-12', 'compras
semanales');
```

Código 3.2.1: Ejemplo simplificado de salida SQL generada.

3.3 Dificultades Encontradas

Una de las principales dificultades fue adaptar un compilador base pensado para una calculadora a un dominio más declarativo y textual. En la calculadora, las expresiones tienen una estructura aritmética tradicional; en este proyecto, en cambio, las sentencias combinan palabras reservadas, campos opcionales, fechas, identificadores y texto libre. Esto obligó a rediseñar tanto la gramática como el AST para representar operaciones heterogéneas.

Otra dificultad fue el tratamiento de palabras en español. El lenguaje debía aceptar mayúsculas, minúsculas y variantes con o sin tilde, sin hacer que la gramática se volviera difícil de leer. La solución fue resolver la mayor parte de este problema en Flex, donde se definieron patrones que reconocen las variantes relevantes antes de enviarlas a Bison como un mismo token.

También resultó delicado el manejo de descripciones. Permitir texto libre sin comillas mejora mucho la usabilidad, pero exige cambiar el contexto léxico para que el scanner no interprete cada palabra de la descripción como un identificador o una palabra reservada. Esta decisión simplifica el input del usuario, aunque impone que la descripción sea el último campo de la línea.

Una dificultad adicional estuvo asociada a la frontera entre validaciones estáticas y validaciones de runtime. Algunas reglas, como montos positivos o fechas válidas, pueden verificarse antes de generar SQL. En cambio, otras reglas dependen de la base de datos, como comprobar si un identificador existe o si corresponde efectivamente a una suscripción o a un plan de cuotas. Por eso, el backend combina validaciones semánticas en C con restricciones expresadas en las sentencias SQL generadas.

Por último, una dificultad importante estuvo asociada al armado de las salidas de reporte, especialmente en formato PDF. La generación de archivos de texto plano, HTML y PDF no formaba parte directa de los contenidos trabajados en la materia ni de los manejos de SQL a los que estábamos acostumbrados. En particular, el PDF exigía resolver problemas de estructura, formato y representación del contenido sin apoyarnos en librerías externas (para evitar problemas de compatibilidad, permitiendo una mayor portabilidad). Por ese motivo, utilizamos herramientas de inteligencia artificial como apoyo para orientar la construcción de estas salidas, manteniendo la implementación dentro del propio proyecto y evitando depender de componentes externos.

4. Futuras Extensiones

Una primera extensión consiste en integrar el compilador con un runtime que ejecute automáticamente el SQL generado contra una base PostgreSQL. En la versión actual, el compilador emite el script por salida estándar, lo cual es simple y flexible, pero requiere que el usuario o una herramienta externa se encargue de ejecutar dicho script.

Otra extensión importante es fortalecer las validaciones referenciales. Actualmente, la existencia de identificadores y la compatibilidad de una operación al finalizarla dependen de la base de datos. Una capa de runtime podría consultar el estado persistido antes de emitir o ejecutar determinadas operaciones, permitiendo mensajes de error más claros para el usuario final.

También se podrían enriquecer los reportes generados, incorporando más información agregada, como totales por categoría, comparaciones entre períodos, evolución temporal de ingresos y gastos, y resúmenes gráficos. Si bien la versión actual ya contempla salidas en texto plano, HTML y PDF, una extensión futura podría mejorar el nivel de detalle y presentación de dichos reportes.

Por último, se podrían mejorar los mensajes de error para que indiquen con mayor precisión el tipo de falla y la ubicación dentro del programa fuente. Esto haría que el lenguaje sea más cómodo de usar, especialmente en programas con varias sentencias, y facilitaría la depuración de entradas inválidas.

5. Conclusiones

El proyecto permitió diseñar e implementar un lenguaje específico de dominio para finanzas personales, aplicando los conceptos de análisis léxico, análisis sintáctico, análisis semántico, construcción de árboles de sintaxis abstracta y generación de código. A partir de una problemática concreta, se definió una sintaxis de alto nivel que permite expresar operaciones financieras de forma breve y cercana al lenguaje natural.

La implementación logró separar claramente las responsabilidades del compilador. El frontend reconoce y estructura el lenguaje, el análisis semántico valida reglas propias del dominio y mantiene contexto mediante una tabla de símbolos, y el generador transforma el AST validado en un script SQL para PostgreSQL. Esta división facilita la evolución del proyecto y permite razonar con claridad sobre cada etapa.

Si bien todavía quedan extensiones posibles, como la ejecución automática sobre PostgreSQL, validaciones referenciales más expresivas y reportes con mayor nivel de análisis, la versión actual cumple con el objetivo principal de modelar un lenguaje propio y demostrar su procesamiento completo mediante Flex, Bison, validaciones semánticas y generación de código. Además, la incorporación de reportes en texto plano, HTML y PDF amplía la utilidad práctica del lenguaje, ya que permite transformar las operaciones registradas en salidas directamente consultables por el usuario.

6. Referencias

- Repositorio del proyecto: https://github.com/facundokrens65049/TPE_TLA
- Especificación inicial del proyecto: *stage1_especificacion.pdf*, ubicada en la carpeta *doc* del repositorio.
- Proyecto base utilizado: *Flex-Bison-Compiler* provisto por la cátedra como estructura inicial para el desarrollo del compilador.
- Archivos principales revisados: *FlexPatterns.l*, *BisonGrammar.y*, *AbstractSyntaxTree.h*, *SemanticAnalyzer.c*, *Generator.c*, *TextReport.c*, *HtmlReport.c*, *PdfReport.c*, *ReportModel.c*, *EntryPoint.c* y casos de prueba ubicados en *src/test/c*.

7. Bibliografía

Golmar, M. A. (2026, 12 de marzo). (2026-03-12) Proyecto Especial [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. <https://docs.google.com/document/d/1XZy63UwUVHyMCR6p051aer-i9SpbLmoOSGC-gh6Cb-I/edit>

Golmar, M. A. (2026, 12 de marzo). (2026-03-12) Modelo Computacional [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. https://docs.google.com/document/d/1V6DsOd9hhpiCoMlqddqVNYaVvWomIZ8ylNW4TD_rruD_Y/edit

Golmar, M. A. (2025, 3 de septiembre). (2025-09-03, v1.0.0) Análisis Léxico [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. <https://docs.google.com/document/d/1o0rYT3vbLtdxuMNnlirf7je2bp2B6FuPGiwCJkVrx0g/edit>

Golmar, M. A. (2024, 8 de mayo). (2024-05-08, v0.1.0) Análisis Sintáctico [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. https://docs.google.com/document/d/1HNNtofu_TCHadDxham2K-_HwKfaCG55izvX8a-QtqZU/edit

Golmar, M. A. (2024, 25 de septiembre). (2024-09-25, v0.2.0) Análisis Semántico [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. https://docs.google.com/document/d/1ZqUt72POtTA-IXPHUVgFqz0hbb7-QQnc5i_Q0B1rMPY/edit

Golmar, M. A. (2024, 16 de mayo). (2024-05-16, v0.1.0) Generación de Código [Material de cátedra]. Autómatas, Teoría de Lenguajes y Compiladores, Instituto Tecnológico de Buenos Aires. https://docs.google.com/document/d/1gTDatettvr3GLewUy4_iwacQvTCZAAks02xu6l6eFcc/edit

Free Software Foundation. (2024). GNU Bison Manual. <https://www.gnu.org/software/bison/manual/>

The Flex Project. (2024). Flex Manual. <https://westes.github.io/flex/manual/>